Reglamento del Torneo Micromouse e-puck

Proyecto final — Proyecto Mecatrónico: Modelado y Simulación, Otoño 2026

Dr. Jorge A. Lizárraga

Versión 1.0 — 10 de noviembre de 2026

Este reglamento establece las condiciones técnicas, la forma de evaluación y el formato de competencia del proyecto final. Se basa en la competencia **Micromouse clásica del IEEE**, adaptada a un robot e-puck simulado en Webots. Todo lo que no esté previsto aquí lo resuelve el profesor (sección 12).

Contenido

1 Objetivo . . . 1
2 Equipos y recursos oficiales . . . 1
3 Definiciones . . . 2
4 Los laberintos . . . 2
4.1 Dimensiones . . . 2
4.2 Reglas que cumplen los laberintos . . . 3
4.3 Laberintos de práctica . . . 3
5 El robot . . . 4
6 El controlador . . . 5
6.1 Estructura . . . 5
6.2 Permitido . . . 5
6.3 Prohibido . . . 5
6.4 Tiempo de ejecución . . . 5
7 Desarrollo de un concurso . . . 5
8 Puntaje . . . 6
9 Formato del torneo . . . 6
9.1 Fase 1: clasificación . . . 7
9.2 Fase 2: eliminatoria uno contra uno . . . 7
10 Entregables y evaluación secuencial . . . 7
10.1 Ponderación . . . 8
11 Sanciones y descalificación . . . 9
12 Situaciones no previstas . . . 9

1 Objetivo

Cada equipo programará en MATLAB un controlador para el e-puck que, **sin conocer el laberinto**, lo explore, construya un mapa, llegue a la meta, regrese al inicio y la alcance de nuevo en el menor tiempo posible. Resolverlo requiere, entre otros temas, el modelo cinemático del robot diferencial, la odometría, el control en lazo cerrado, el procesamiento de sensores y la planeación de rutas. Investigar y aplicar los temas necesarios forma parte del proyecto y es responsabilidad de cada equipo.

2 Equipos y recursos oficiales

- Equipos de **máximo 3 integrantes**, inscritos según la convocatoria.
- Simulador: **Webots R2025a**. Lenguaje: **MATLAB**, únicamente con funciones de MATLAB base (sin toolboxes).
- El profesor distribuye un repositorio oficial con:
  - 20 laberintos de práctica listos para usarse, 10 de 8×8 y 10 de 16×16 (`worlds/`);
  - el árbitro automático (`controllers/referee/`);
  - una plantilla de controlador (`controllers/epuck_plantilla/`);
  - las piezas del laberinto (`protos/`).

- **Los equipos no diseñan laberintos**: el único entregable de programación es el controlador del robot.
- Los archivos oficiales **no se modifican**. En la competencia se usan las versiones del profesor.

3 Definiciones

Celda: cuadro de 18×18 cm delimitado por cuatro postes.

Inicio: la celda de la esquina suroeste, $(0, 0)$. El robot parte de su centro, viendo al norte.

Meta: el bloque de 2×2 celdas del centro del laberinto. Su piso es amarillo.

Corrida: recorrido que empieza cuando el centro del robot sale de la celda de inicio y termina cuando entra a la meta.

Concurso: el turno completo de un robot en un laberinto. Dura como máximo 10 minutos de simulación y puede incluir varias corridas.

Tiempo de simulación: el reloj interno de Webots. Es el único que cuenta, de modo que el resultado no depende de la velocidad de la computadora.

Árbitro: el controlador Supervisor oficial que mide tiempos, cuenta corridas y registra los resultados.

4 Los laberintos

La competencia se corre en laberintos **sorpresa** de 16×16, diseñados por el profesor, que nadie conoce antes del torneo. Para desarrollar y probar, el repositorio incluye 20 laberintos de práctica: 10 de 8×8 y 10 de 16×16. **Los equipos no diseñan laberintos.**

4.1 Dimensiones

Elemento	Medida
Celda (distancia entre centros de postes)	18 cm
Pared	16.8 cm de largo, 1.2 cm de grosor, 5 cm de alto
Poste	$1{,}2 \times 1{,}2$ cm, 5 cm de alto
Color	Paredes blancas con tapa roja; piso negro; meta amarilla
Laberinto de competencia	16×16 celdas ($2{,}88 \times 2{,}88$ m)
Laberintos de práctica	8×8 y 16×16 (incluidos en el repositorio)

Cuadro 1: Dimensiones del laberinto (estándar Micromouse clásico del IEEE).

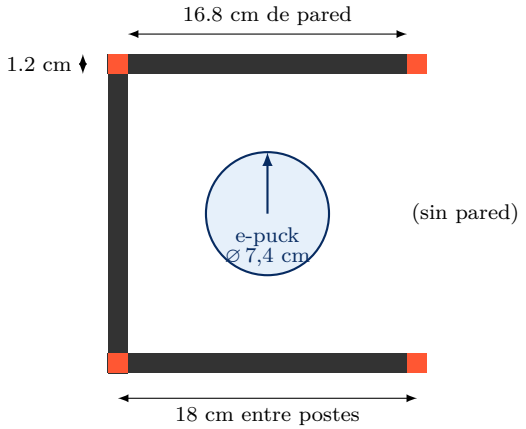


Figura 1: Una celda vista desde arriba, con el e-puck al centro. Con el robot centrado, cada pared lateral queda a unos 5.3 cm.

4.2 Reglas que cumplen los laberintos

Los laberintos de práctica y los de competencia los genera el profesor, y todos cumplen las siguientes reglas. El controlador puede suponerlas, pero no debe suponer nada más del laberinto.

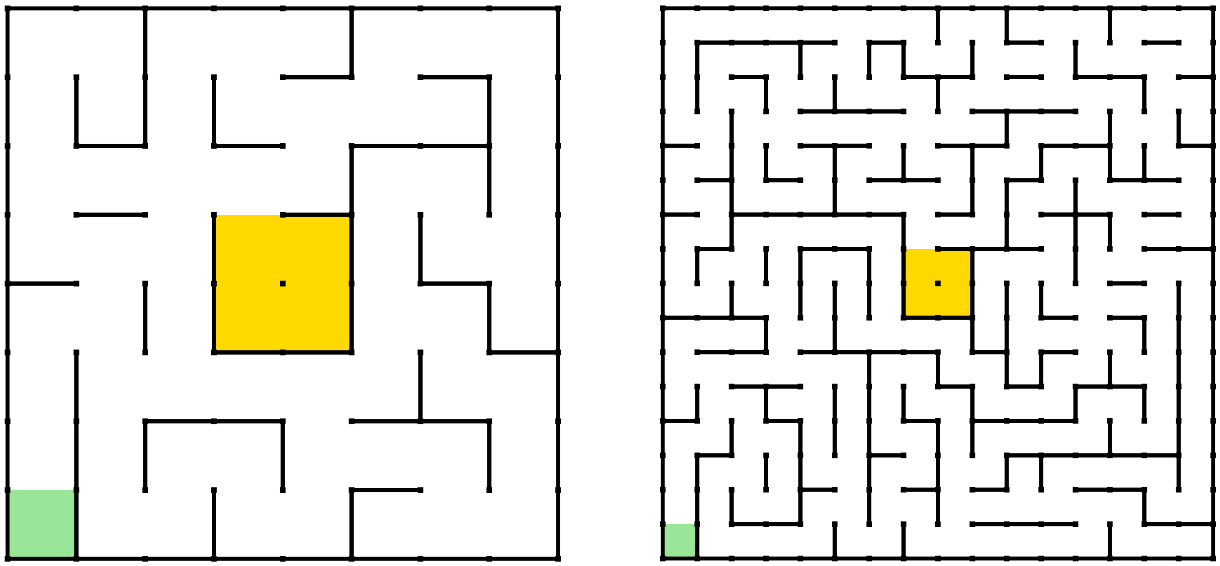
Regla	Descripción
R1	Cuadrado de $N \times N$ celdas, con N par. En competencia, $N = 16$.
R2	Paredes consistentes entre celdas vecinas.
R3	Perímetro completamente cerrado.
R4	El inicio es la esquina suroeste, cerrado por tres lados (oeste, sur y este) y abierto solo al norte.
R5	La meta es el bloque central de 2×2 celdas, sin paredes internas y con **una sola entrada**.
R6	Todo poste, excepto el poste central de la meta, toca al menos una pared.
R7	La meta es alcanzable desde el inicio.
R8	La meta **no** se alcanza siguiendo la pared derecha ni siguiendo la pared izquierda.

Cuadro 2: Reglas de los laberintos.

Los laberintos sorpresa tienen además **varias rutas** a la meta (ciclos): la primera ruta que encuentre el robot no necesariamente es la más corta.

4.3 Laberintos de práctica

El repositorio incluye 20 mundos listos para usarse: `worlds/practica_8x8_01.wbt` a `practica_8x8_10.wbt`, para desarrollar, y `worlds/practica_16x16_01.wbt` a `practica_16x16_10.wbt`, para verificar en el tamaño de competencia. La figura 2 muestra los dos primeros. Todos cumplen las reglas R1–R8 y cada uno tiene una ruta y unos ciclos distintos. Un controlador que solo funciona en estos laberintos **no** necesariamente funcionará en la competencia.



(a) 8×8 (b) 16×16

Figura 2: Laberintos de práctica vistos desde arriba, con el norte hacia arriba. Inicio en verde y meta en amarilla.

5 El robot

El robot es el **e-puck estándar de GCtronic** (versión 1) con la extensión de sensores de piso, tal como lo definen los mundos oficiales. El paso de simulación es de 16 ms y la velocidad máxima de cada rueda es de 6.28 rad/s. El robot no se modifica.

Dispositivo	**Nombre en Webots**
Motores de rueda	`left wheel motor`, `right wheel motor`
Encoders	`left wheel sensor`, `right wheel sensor`
Sensores infrarrojos de proximidad	`ps0` … `ps7`
Sensores de luz	`ls0` … `ls7`
Sensores de piso	`gs0`, `gs1`, `gs2`
Acelerómetro y giroscopio	`accelerometer`, `gyro`
Cámara (52×39 píxeles)	`camera`
LEDs	`led0` … `led9`
Emisor y receptor	`emitter`, `receiver`

Cuadro 3: Dispositivos del e-puck. La plantilla oficial ya los inicializa todos.

Uso de los dispositivos. El proyecto exige aprovechar las capacidades del e-puck, como mínimo:

- los **sensores infrarrojos**, para detectar paredes y centrarse en el pasillo;
- los **encoders**, para la odometría;
- el **giroscopio** o el **acelerómetro**, para corregir giros o detectar choques;
- los **sensores de piso**, para confirmar la llegada a la meta;
- la **cámara**, con un uso justificado en el reporte;
- los **LEDs**, para indicar el estado del robot (explorando, regresando, corrida rápida, meta).

El uso de cada dispositivo se documenta y se evalúa en el reporte técnico (sección 10).

6 El controlador

6.1 Estructura

- El controlador es una carpeta `controllers/<equipo>/` cuyo archivo principal es `<equipo>.m`. La carpeta y el archivo deben llamarse igual.
- El nombre del equipo usa solo minúsculas, dígitos y guion bajo, sin espacios ni acentos, y tiene como máximo 20 caracteres. Ejemplo: `equipo_rojo`.
- Se pueden agregar funciones auxiliares (`.m`) dentro de la misma carpeta.
- El controlador debe ejecutarse sin intervención: no pide datos al usuario ni espera teclas.
- Para probarlo en los laberintos de práctica: en el árbol de la escena de Webots se selecciona `DEF MOUSE E-puck`, se elige la carpeta del equipo en su campo `controller` y se guarda el mundo. Es el único cambio permitido a los mundos.

6.2 Permitido

- Leer cualquier dispositivo del e-puck y controlar sus motores y LEDs.
- Guardar en memoria el mapa construido y usarlo en las corridas siguientes **del mismo concurso**.
- Imprimir en la consola y escribir archivos de registro dentro de su propia carpeta, para depuración.

6.3 Prohibido

- Usar funciones de Supervisor (`wb_supervisor_*`) o cualquier forma de conocer la posición absoluta del robot.
- Leer archivos de laberinto, mundos (`.wbt`), resultados del árbitro o **cualquier archivo de datos**, incluidos los generados en concursos anteriores. El robot debe empezar cada concurso sin información previa del laberinto.
- Incluir en el controlador laberintos guardados, precalculados o generados con las herramientas del repositorio, incluidos los de práctica, así como rutas fijas.
- Recibir información externa mediante el receptor o por cualquier otro medio.
- Modificar el robot, el mundo, los PROTO, el árbitro o la configuración de la simulación.
- Usar toolboxes de MATLAB, archivos MEX o programas externos.

6.4 Tiempo de ejecución

Los resultados se miden en tiempo de simulación, pero el profesor corre todos los controladores en su equipo. Si un concurso completo (10 minutos de simulación) tarda más de **40** minutos reales, se detiene y se toman los resultados obtenidos hasta ese momento.

7 Desarrollo de un concurso

1. El robot se coloca en el centro de la celda de inicio, viendo al norte, y la simulación empieza en $t = 0$.
2. Una **corrida** inicia cuando el centro del robot **sale** de la celda de inicio y termina cuando **entra** a cualquiera de las cuatro celdas de la meta.
3. Para iniciar otra corrida, el robot debe regresar **por sí solo** a la celda de inicio. El regreso no se cronometra, aunque sí consume tiempo del concurso.

4. Si durante una corrida el robot vuelve a la celda de inicio sin haber llegado a la meta, esa corrida se **anula**. Puede intentar otra.
5. No hay intervención humana: el robot no se toca, no se reinicia y no se reacomoda durante el concurso.
6. El concurso termina con lo que ocurra primero:
   - se cumplen **10 minutos** de simulación;
   - el robot **no cambia de celda durante 60 s** (se considera atorado);
   - el robot permanece **30 s en la celda de inicio** después de haber completado al menos una corrida. Así el equipo puede terminar su turno antes.
7. Si el controlador se detiene por un error de MATLAB, los motores conservan la última velocidad ordenada y el concurso continúa hasta que se cumple alguna de las condiciones anteriores. Se conservan las corridas ya completadas.
8. El árbitro muestra el marcador en pantalla y registra el resultado en `results/resultados.csv`. **El registro del árbitro es el resultado oficial.**

8 Puntaje

Cada corrida completa i recibe el puntaje

$$P_i = t_{\text{corrida},i} + \frac{t_{\text{salida},i}}{30},$$

donde $t_{\text{corrida},i}$ es la duración de la corrida y $t_{\text{salida},i}$ es el instante, contado desde $t = 0$, en que el robot salió del inicio para esa corrida. El segundo término penaliza, de forma moderada, el tiempo usado antes de la corrida, de modo que explorar con rapidez también cuenta. El **puntaje del concurso** es el menor P_i; gana quien tenga el puntaje **más bajo**.

Ejemplo. Un robot explora y llega a la meta por primera vez a los 95 s (corrida 1, que salió en $t = 0{,}4$ s). Regresa al inicio y sale de nuevo en $t = 150$ s, y llega a la meta en $t = 172$ s (corrida 2). Entonces $P_1 = 94{,}6 + 0{,}4/30 \approx 94{,}61$ y $P_2 = 22 + 150/30 = 27$. Su puntaje es 27.

Robots que no llegan a la meta. Como $t_{\text{salida},i} + t_{\text{corrida},i} \leq 600$ s, una corrida completa nunca da más de 600. Si el robot no completa ninguna corrida, el árbitro le asigna el puntaje

$$P = 600 + 30\,d + \frac{t_d}{30},$$

donde d es la **distancia a la meta**, en celdas y por los pasillos del laberinto, de la celda más cercana a la meta que visitó el robot, y t_d es el instante en que llegó a esa celda por primera vez. Así, llegar a la meta siempre es mejor que no llegar, y entre los robots que no llegan queda arriba el que se acercó más.

Ejemplo. Un robot no llega a la meta. La celda más cercana que visitó está a 3 celdas de la meta y llegó a ella en $t = 420$ s. Su puntaje es $P = 600 + 30(3) + 420/30 = 704$.

Orden y desempate. Los robots se ordenan con los siguientes criterios, en este orden:

1. menor puntaje del concurso;
2. en un empate exacto, mayor número de **celdas distintas visitadas**, que registra el árbitro;
3. mejor posición en la fase de clasificación (solo en la eliminatoria).

9 Formato del torneo

Todos los laberintos del torneo son **sorpresa**: son de 16×16, cumplen las reglas R1–R8, los diseña el profesor y **se mantienen en secreto** hasta después del torneo. Cada robot corre **solo** en su simulación; los enfrentamientos uno contra uno se deciden comparando los resultados de cada robot en el mismo laberinto.

9.1 Fase 1: clasificación

Cada robot realiza un concurso en el **laberinto de clasificación**. Todos los robots corren el mismo laberinto, para que sus puntajes se puedan comparar. Con el orden de la sección 8 se obtiene la tabla de clasificación, que define las posiciones de la siguiente fase.

9.2 Fase 2: eliminatoria uno contra uno

- Las llaves se arman por posición: el 1.ř contra el último, el 2.ř contra el penúltimo, y así sucesivamente. Si el número de equipos no es potencia de 2, los mejor clasificados pasan directo a la segunda ronda.
- **Cada llave tiene su propio laberinto secreto**, distinto de todos los demás del torneo, incluso de los de otras llaves de la misma ronda. Como la simulación es determinista, repetir un laberinto daría el mismo resultado.
- En cada duelo, los dos robots corren un concurso en el laberinto de su llave, uno después del otro. Corre primero el mejor clasificado. Gana el de menor puntaje, con los mismos criterios de desempate.
- Los perdedores de las semifinales disputan el tercer lugar en un laberinto propio.

Al terminar el torneo se publican los laberintos sorpresa y los resultados de cada concurso.

10 Entregables y evaluación secuencial

Cada entrega se sube a la plataforma del curso como un archivo `<equipo>.zip` que contiene la carpeta `controllers/<equipo>/` y los documentos que pida cada hito.

> **La evaluación es secuencial.** Las entregas forman una lista que se cumple en orden. Una entrega **solo se evalúa si todas las anteriores fueron entregadas**. Si un equipo no entrega un hito, **ninguna de las entregas siguientes se evalúa**, incluidas la competencia y el reporte técnico. Por ejemplo, si no se entrega el H2, no se evalúan el H3, el H4, la competencia ni el reporte.
> **Todas las entregas se concentran en las semanas 14 y 15**: los hitos H1–H4 se entregan junto con el controlador final, el martes 17 de noviembre, y cada hito se evalúa en el orden de la lista.

Entrega del controlador: martes 17 de noviembre (semana 14)

El controlador se entrega **una semana antes del torneo** para resolver a tiempo **cualquier percance**: errores de MATLAB, nombres de carpeta o problemas de compatibilidad con el equipo del profesor. Después del 17 de noviembre **no se recibe ningún controlador**. Hasta el viernes 20 de noviembre solo se aceptan correcciones de compatibilidad sobre el controlador ya entregado, no mejoras.

- [ ] **1. Inscripción** — **semana 14, en el salón de clases**
  Nombre del equipo e integrantes.
- [ ] **2. H1 — Sensores y modelo** — **martes 17 de noviembre (semana 14)**
  Calibración de los sensores infrarrojos (gráfica de lectura contra distancia) y modelo cinemático del e-puck con su odometría.
- [ ] **3. H2 — Movimiento celda por celda** — **martes 17 de noviembre (semana 14)**
  En el laberinto de 8×8: avanzar una celda, girar $\pm 90°$ y $180°$ y centrarse en el pasillo, con el error medido.
- [ ] **4. H3 — Mapeo y flood fill** — **martes 17 de noviembre (semana 14)**
  El robot llega a la meta en los laberintos de 8×8 y de 16×16.
- [ ] **5. H4 — Robot completo** — **martes 17 de noviembre (semana 14)**
  Regreso al inicio, corridas rápidas y uso de todos los dispositivos.

☐ **6. Entrega del controlador final** **martes 17 de noviembre (semana 14)**
Carpeta `controllers/<equipo>/` definitiva. Esa semana el profesor la corre en su equipo, en el laberinto de práctica de 16 × 16, para revisar que funcione y que no haya problemas de compatibilidad. El resultado se publica, pero no cuenta para la calificación. Después de esta revisión solo se aceptan correcciones de compatibilidad (nombres, rutas o errores de MATLAB), no mejoras, a más tardar el **viernes 20 de noviembre**.

☐ **7. Torneo** **semana 15, del 23 al 27 de noviembre**
Clasificación y eliminatoria uno contra uno en laberintos sorpresa.

☐ **8. Reporte técnico y video** **semana 15, del 23 al 27 de noviembre**
Reporte en PDF y video de 3 minutos como máximo.

Reporte técnico. Se elabora en LaTeX con la plantilla **BitacoraSKv2**. La plantilla viene lista para usarse en la carpeta `docs/bitacora/plantilla/` del repositorio oficial: se trabaja sobre `main.tex`. El **Manual de Uso BitacoraSKv2**, en `docs/bitacora/`, explica cómo usarla. El reporte debe incluir:

- el modelo cinemático del e-puck y la odometría;
- la calibración de los sensores;
- el diseño del control (movimiento y centrado);
- el algoritmo de exploración y de cálculo de rutas;
- el uso de cada dispositivo;
- los resultados en al menos un laberinto de práctica de 8 × 8 y uno de 16 × 16 (tiempos de exploración y de corrida rápida);
- las conclusiones.

10.1 Ponderación

Rubro	Peso
Hitos H1–H4 (**10** cada uno)	**40 %**
Competencia	**30 %**
Reporte técnico y video	**30 %**

Cuadro 4: Ponderación del proyecto final. Cada rubro se evalúa solo si se cumplen las entregas anteriores de la lista.

Competencia. **15 %** se obtiene al completar al menos una corrida en el laberinto de clasificación. El **15 %** restante depende del lugar final:

- 1.er lugar: 15 %;
- 2.ř lugar: 13 %;
- 3.er lugar: 12 %;
- 4.ř lugar: 11 %;
- eliminados en cuartos de final: 9 %;
- resto: 7 %;
- robots sin ninguna corrida completa: 0 %.

Los porcentajes son de la calificación del proyecto final, y la competencia suma como máximo 30 %: hasta 15 % por completar una corrida en la clasificación más hasta 15 % por el lugar final.

Ejemplo 10.1:

Supongamos un torneo de 8 equipos, con cuartos de final, semifinal, final y partido por el tercer lugar.

- **Equipo A.** Completa una corrida en la clasificación: 15 %. Gana en cuartos, pierde en semifinal y gana el partido por el tercer lugar: 3.er lugar, 12 %. Competencia: $15\,\% + 12\,\% = 27\,\%$ de 30 %.
- **Equipo B.** Completa una corrida en la clasificación: 15 %. Pierde en cuartos de final: 9 %. Competencia: $15\,\% + 9\,\% = 24\,\%$ de 30 %.
- **Equipo C.** No completa ninguna corrida en la clasificación: 0 %. Queda último y, en cuartos, completa una corrida pero pierde: 9 %. Competencia: $0\,\% + 9\,\% = 9\,\%$ de 30 %.
- **Equipo D.** No completa ninguna corrida ni en la clasificación ni en cuartos: $0\,\% + 0\,\% = 0\,\%$.

Si el Equipo A obtiene además 35 % en los hitos (de 40 %) y 25 % en el reporte y el video (de 30 %), su calificación del proyecto es $35\,\% + 27\,\% + 25\,\% = 87\,\%$.

11 Sanciones y descalificación

- Violar cualquier punto de la sección “Prohibido” o modificar archivos oficiales **descalifica al equipo** del torneo. La calificación de competencia es 0, sin perjuicio de las sanciones académicas que correspondan.
- Un controlador que no corre en el equipo del profesor por errores de nombre o estructura y que no se corrigió después de la prueba de compatibilidad compite con lo que logre hacer en ese estado.
- Copiar código de otro equipo, total o parcialmente, se considera falta de integridad académica.

12 Situaciones no previstas

Cualquier situación que no esté prevista en este reglamento la resuelve el profesor. Las aclaraciones sobre un resultado se presentan por escrito dentro de las 24 horas siguientes a su publicación. Si un problema del equipo del profesor afecta un concurso, ese concurso se repite.